

Blocks

Set Wheel Speed

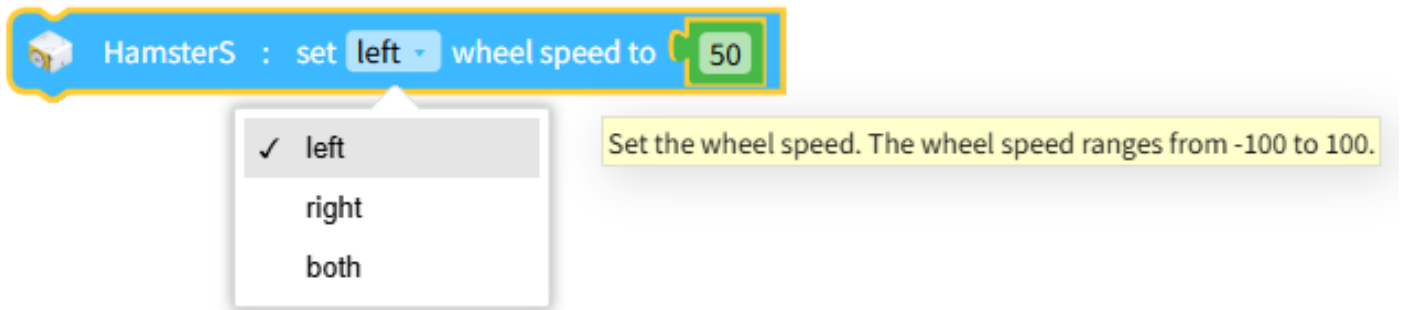
Sets the wheel speed of Hamster S.

If the wheel speed is positive, it rotates forward, and if the wheel speed is negative, it rotates backward.

For example, if the wheel speed is 100, it rotates forward at speed 100,

and if the wheel speed is -100, it rotates backward at speed 100.

Once you set the wheel speed, Hamster S will keep moving at that speed until you set the wheel speed again.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
wheel	Dropdown Option	Wheel type	left, right, both
velocity	Input Value	Wheel speed	Integer -100 ~ 100, 0: stop

JavaScript Code

```
// Set left wheel speed to 50
if($('HamsterS*0:wheel.move').d != 0) {
    $('HamsterS*0:wheel.move').d = 0;
}
$('HamsterS*0:wheel.speed.left').d = __getSpeed('HamsterS*0', 50);

// Set right wheel speed to 50
if($('HamsterS*0:wheel.move').d != 0) {
    $('HamsterS*0:wheel.move').d = 0;
}
```

```
}
$( 'HamsterS*0:wheel.speed.right' ).d = __getSpeed( 'HamsterS*0', 50 );
```

Python Code

```
# Set both wheel speeds to 50
if __('HamsterS*0:wheel.move').d != 0:
    __('HamsterS*0:wheel.move').d = 0
__('HamsterS*0:wheel.speed.left').d = __getSpeed( 'HamsterS*0', 50)
__('HamsterS*0:wheel.speed.right').d = __getSpeed( 'HamsterS*0', 50)
```

Move by Distance

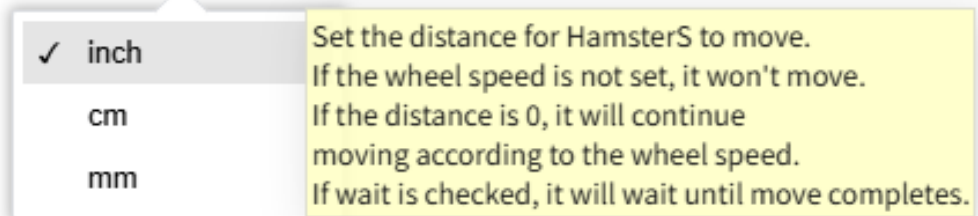
Sets the distance Hamster S will move.

If wheel speed is not set, it will not move.

If the distance value is 0, it keeps moving without stopping at the currently set wheel speed.

If you check “wait”, it waits until the move is completed.

If you check “wait”, it can only be used inside an async function.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
distance	Input Value	Distance value	Real number $\neq 0$
unit	Dropdown Option	Distance unit	cm, mm, inch

JavaScript Code

```
// Move 5 cm | Wait 0
$('HamsterS*0:wheel.move').d = __getDistance('HamsterS*0', 5, 'cm'); // (robot,
distance, unit)
await $('HamsterS*0:wheel.!move').w();

// Move 1000 mm | Wait X
$('HamsterS*0:wheel.move').d = __getDistance('HamsterS*0', 1000, 'mm'); //
(robot, distance, unit)
```

Python Code

```
# Move 5 cm | Wait 0
__('HamsterS*0:wheel.move').d = __getDistance('HamsterS*0', 5, 'cm') # (robot,
distance, unit)
await __('HamsterS*0:wheel.!move').w()

# Move 1000 mm | Wait X
__('HamsterS*0:wheel.move').d = __getDistance('HamsterS*0', 1000, 'mm') #
(robot, distance, unit)
```

Move by Time

Sets the time Hamster S will move.

If wheel speed is not set, it will not move.

If you check “wait”, it waits until the move is completed.

If you check “wait”, it can only be used inside an async function.



Set the time for HamsterS to move.
If the wheel speed is not set, it won't move.
If wait is checked, it will wait until move completes.

Dropdown Options and Input Values

Name	Type	Description	Range / Type
time	Input Value	Time value	Real number ≥ 0
unit	Dropdown Option	Time unit	seconds, milliseconds

If you set the option to milliseconds, the input will be the value of time divided by 1000.

JavaScript Code

```
// Move 5 seconds | Wait 0
await __stopAfterDelay('HamsterS*0', 5, true); // (robot, time, wait_w)

// Move 5000 milliseconds | Wait X
__stopAfterDelay('HamsterS*0', 5, false); // (robot, time, wait_w)
```

Python Code

```
# Move 5 seconds | Wait 0
await __stopAfterDelay('HamsterS*0', 5, True) # (robot, time, wait_w)
```

```
# Move 5000 milliseconds | Wait X
__stopAfterDelay('HamsterS*0', 5, False) # (robot, time, wait_w)
```

Turn in Place

Sets the direction and angle for Hamster S to rotate in place.

If you check “wait”, it waits until the move is completed.

If you check “wait”, it can only be used inside an async function.



Set the direction and degrees to turn in place.
If wait is checked, it will wait until move completes.

Dropdown Options and Input Values

Name	Type	Description	Range / Type
direction	Dropdown Option	Turn direction	left, right
degree	Input Value	Turn angle	Integer ≥ 0

JavaScript Code

```
// Turn left 90 degrees | Wait 0
await __turn_degree_left('HamsterS*0', 90, true); // (robot, degree, wait_w)

// Turn right 270 degrees | Wait X
__turn_degree_right('HamsterS*0', 270, false); // (robot, degree, wait_w)
```

Python Code

```
# Turn left 90 degrees | Wait 0
await __turn_degree_left('HamsterS*0', 90, True) # (robot, degree, wait_w)

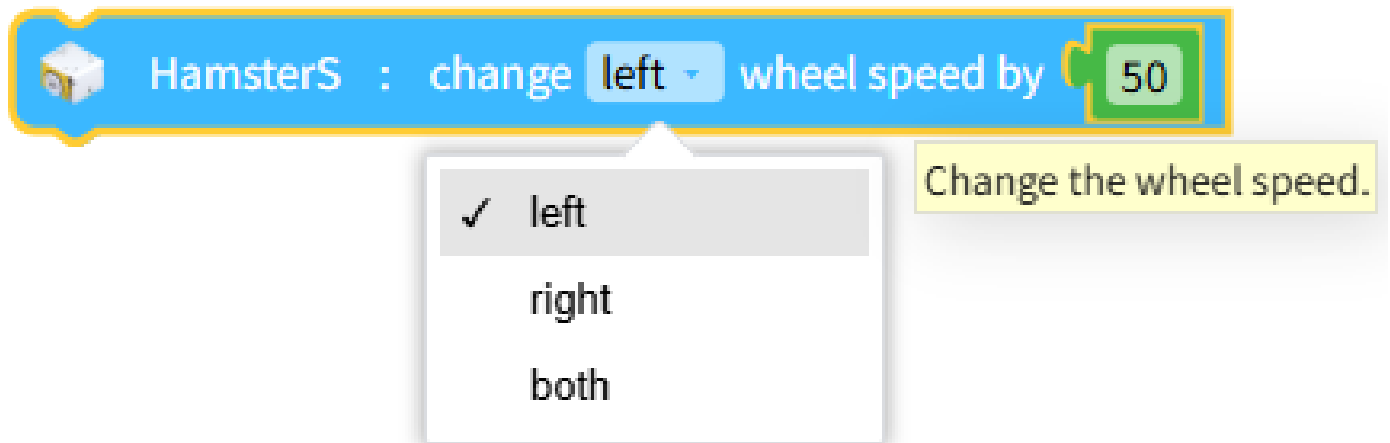
# Turn right 270 degrees | Wait X
__turn_degree_right('HamsterS*0', 270, False) # (robot, degree, wait_w)
```

Change Wheel Speed

Changes the wheel speed of Hamster S.

The new wheel speed becomes the current wheel speed plus the input value.

The new wheel speed is clamped to the range -100 ~ 100.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
wheel	Dropdown Option	Wheel type	left, right, both
velocity	Input Value	Value to add to current speed	Integer -200 ~ 200, 0: stop

JavaScript Code

```
// Change left wheel speed by +50
if($('HamsterS*0:wheel.move').d != 0) {
```

```

    $('HamsterS*0:wheel.move').d = 0;
}
$('HamsterS*0:wheel.speed.left').d = $('HamsterS*0:wheel.speed.left').d +
__getSpeed('HamsterS*0', 50);

// Change right wheel speed by +50
if($('HamsterS*0:wheel.move').d != 0) {
    $('HamsterS*0:wheel.move').d = 0;
}
$('HamsterS*0:wheel.speed.right').d = $('HamsterS*0:wheel.speed.right').d +
__getSpeed('HamsterS*0', 50);

// Change both wheel speeds by +50
if($('HamsterS*0:wheel.move').d != 0) {
    $('HamsterS*0:wheel.move').d = 0;
}
$('HamsterS*0:wheel.speed.left').d = $('HamsterS*0:wheel.speed.left').d +
__getSpeed('HamsterS*0', 50);
$('HamsterS*0:wheel.speed.right').d = $('HamsterS*0:wheel.speed.right').d +
__getSpeed('HamsterS*0', 50);

```

Python Code

```

# Change left wheel speed by +50
if __('HamsterS*0:wheel.move').d != 0:
    __('HamsterS*0:wheel.move').d = 0
__('HamsterS*0:wheel.speed.left').d = __('HamsterS*0:wheel.speed.left').d +
__getSpeed('HamsterS*0', 50)

# Change right wheel speed by +50
if __('HamsterS*0:wheel.move').d != 0:
    __('HamsterS*0:wheel.move').d = 0
__('HamsterS*0:wheel.speed.right').d = __('HamsterS*0:wheel.speed.right').d +
__getSpeed('HamsterS*0', 50)

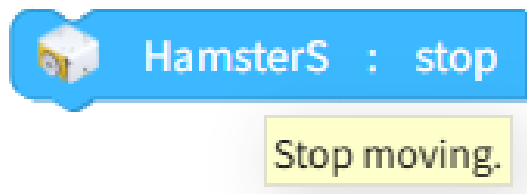
```

```
# Change both wheel speeds by +50
if __('HamsterS*0:wheel.move').d != 0:
    __('HamsterS*0:wheel.move').d = 0
__('HamsterS*0:wheel.speed.left').d = __('HamsterS*0:wheel.speed.left').d +
__getSpeed('HamsterS*0', 50)
__('HamsterS*0:wheel.speed.right').d = __('HamsterS*0:wheel.speed.right').d +
__getSpeed('HamsterS*0', 50)
```

Stop

Stops Hamster S movement.

Both wheel speeds are reset to 0.



JavaScript Code

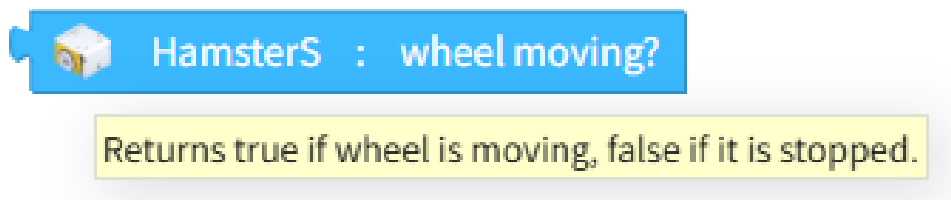
```
__stopMove('HamsterS*0');
```

Python Code

```
__stopMove('HamsterS*0')
```

Are the Wheels Moving?

Returns whether Hamster S wheels are moving as **True(1)** / **False(0)**.



JavaScript Code

```
$( 'HamsterS*0:wheel.moving' ).d
```

Python Code

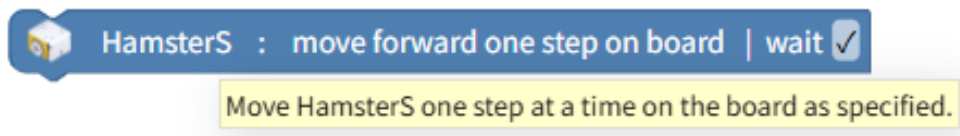
```
__( 'HamsterS*0:wheel.moving' ).d
```

Move One Tile Forward on the Board

Moves Hamster S forward by one tile on the board.

If you check “wait”, it waits until the move is completed.

If you check “wait”, it can only be used inside an async function.



JavaScript Code

```
// Move one tile forward | Wait 0
await __grid_move_forward('HamsterS*0', true); // (robot, wait_w)

// Move one tile forward | Wait X
__grid_move_forward('HamsterS*0', false); // (robot, wait_w)
```

Python Code

```
# Move one tile forward | Wait 0
await __grid_move_forward('HamsterS*0', True) # (robot, wait_w)

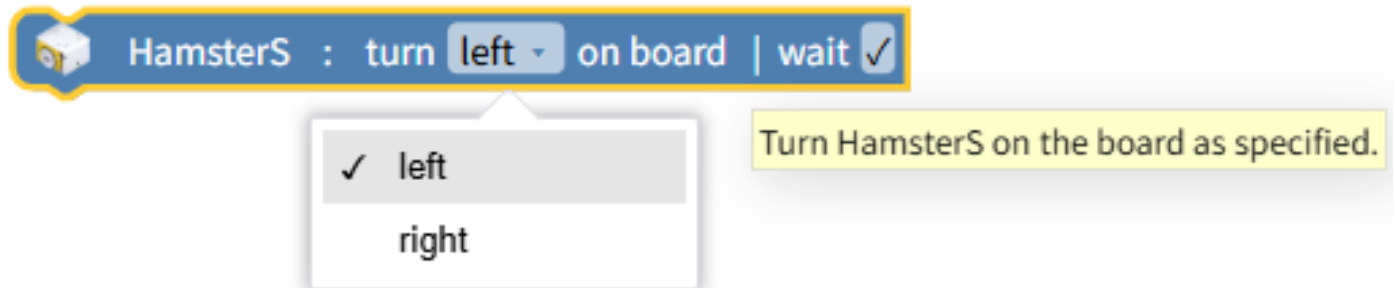
# Move one tile forward | Wait X
__grid_move_forward('HamsterS*0', False) # (robot, wait_w)
```

Turn Once on the Board

Rotates Hamster S 90 degrees on the board in the given direction.

If you check “wait”, it waits until the move is completed.

If you check “wait”, it can only be used inside an async function.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
direction	Dropdown Option	Turn direction	left, right

JavaScript Code

```
// Turn left once on the board | Wait 0
await __grid_turn_left('HamsterS*0', true); // (robot, wait_w)

// Turn left once on the board | Wait X
__grid_turn_left('HamsterS*0', false); // (robot, wait_w)

// Turn right once on the board | Wait 0
await __grid_turn_right('HamsterS*0', true); // (robot, wait_w)

// Turn right once on the board | Wait X
__grid_turn_right('HamsterS*0', false); // (robot, wait_w)
```

Python Code

```
# Turn left once on the board | Wait 0
await __grid_turn_left('HamsterS*0', True) # (robot, wait_w)
```

```

# Turn left once on the board | Wait X
__grid_turn_left('HamsterS*0', False) # (robot, wait_w)

# Turn right once on the board | Wait 0
await __grid_turn_right('HamsterS*0', True) # (robot, wait_w)

# Turn right once on the board | Wait X
__grid_turn_right('HamsterS*0', False) # (robot, wait_w)

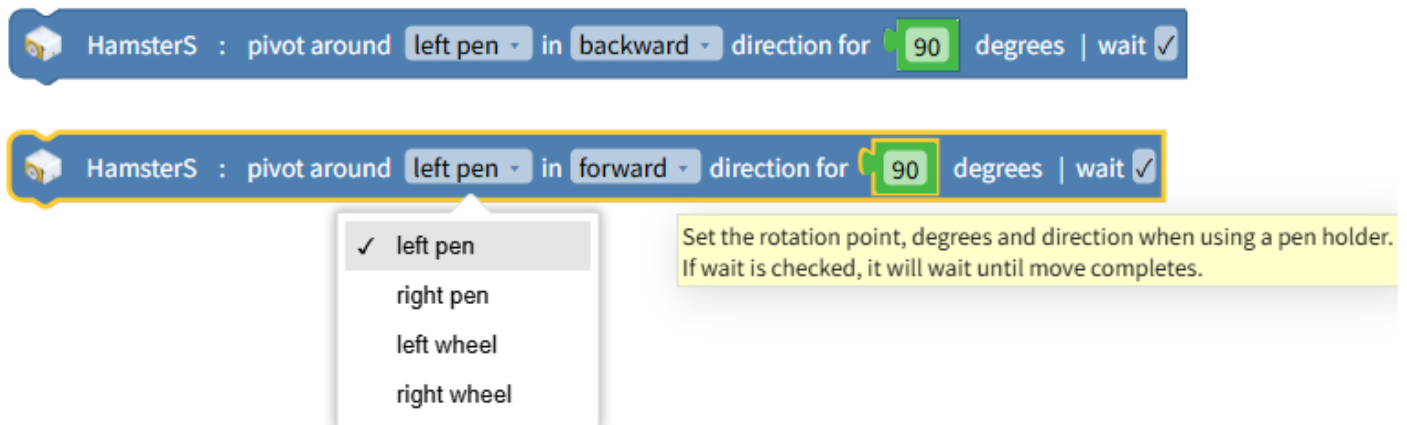
```

Pivot Turn (Pen Holder Reference)

While using the pen holder, sets the pivot reference and direction/degree for Hamster S to rotate in place.

If you check “wait”, it waits until the move is completed.

If you check “wait”, it can only be used inside an async function.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
pivot	Dropdown Option	Pivot reference	left pen, right pen, left wheel, right wheel
direction	Dropdown Option	Direction	forward, backward
degree	Input Value	Rotation angle	Integer ≥ 0

JavaScript Code

```
// Pivot on left pen, forward, 90 degrees | Wait 0
await __pivot('HamsterS*0', 'left pen', 'forward', 90, true); //(robot, pivot,
direction, degree, wait_w)

// Pivot on left pen, backward, 90 degrees | Wait X
__pivot('HamsterS*0', 'left pen', 'backward', 90, false); //(robot, pivot,
direction, degree, wait_w)

// Pivot on right pen, forward, 90 degrees | Wait 0
await __pivot('HamsterS*0', 'right pen', 'forward', 90, true); //(robot, pivot,
direction, degree, wait_w)

// Pivot on right pen, backward, 90 degrees | Wait X
__pivot('HamsterS*0', 'right pen', 'backward', 90, false); //(robot, pivot,
direction, degree, wait_w)

// Pivot on left wheel, forward, 90 degrees | Wait 0
await __pivot('HamsterS*0', 'left wheel', 'forward', 90, true); //(robot, pivot,
direction, degree, wait_w)

// Pivot on left wheel, backward, 90 degrees | Wait X
__pivot('HamsterS*0', 'left wheel', 'backward', 90, false); //(robot, pivot,
direction, degree, wait_w)

// Pivot on right wheel, forward, 90 degrees | Wait 0
await __pivot('HamsterS*0', 'right wheel', 'forward', 90, true); //(robot, pivot,
direction, degree, wait_w)

// Pivot on right wheel, backward, 90 degrees | Wait X
__pivot('HamsterS*0', 'right wheel', 'backward', 90, false); //(robot, pivot,
direction, degree, wait_w)
```

Python Code

```
# Pivot on left pen, forward, 90 degrees | Wait 0
await __pivot('HamsterS*0', 'left pen', 'forward', 90, True) # (robot, pivot,
direction, degree, wait_w)

# Pivot on left pen, backward, 90 degrees | Wait X
__pivot('HamsterS*0', 'left pen', 'backward', 90, False) # (robot, pivot,
direction, degree, wait_w)

# Pivot on right pen, forward, 90 degrees | Wait 0
await __pivot('HamsterS*0', 'right pen', 'forward', 90, True) # (robot, pivot,
direction, degree, wait_w)

# Pivot on right pen, backward, 90 degrees | Wait X
__pivot('HamsterS*0', 'right pen', 'backward', 90, False) # (robot, pivot,
direction, degree, wait_w)

# Pivot on left wheel, forward, 90 degrees | Wait 0
await __pivot('HamsterS*0', 'left wheel', 'forward', 90, True) # (robot, pivot,
direction, degree, wait_w)

# Pivot on left wheel, backward, 90 degrees | Wait X
__pivot('HamsterS*0', 'left wheel', 'backward', 90, False) # (robot, pivot,
direction, degree, wait_w)

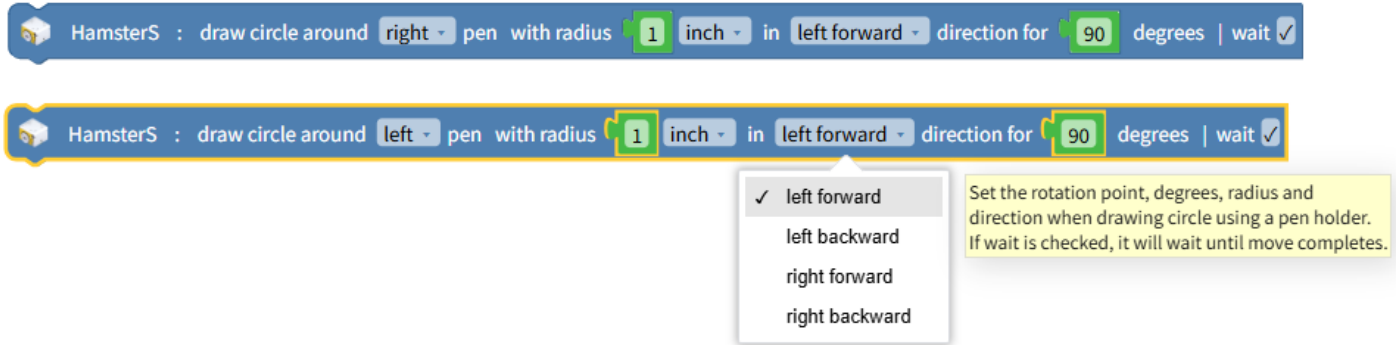
# Pivot on right wheel, forward, 90 degrees | Wait 0
await __pivot('HamsterS*0', 'right wheel', 'forward', 90, True) # (robot, pivot,
direction, degree, wait_w)

# Pivot on right wheel, backward, 90 degrees | Wait X
__pivot('HamsterS*0', 'right wheel', 'backward', 90, False) # (robot, pivot,
direction, degree, wait_w)
```

Pivot Circle (Draw a Circle with Pen Holder)

While using the pen holder, sets pivot, direction, radius, and degree for Hamster S to draw a circular path. If you check “wait”, it waits until the move is completed.

If you check “wait”, it can only be used inside an async function.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
pivot	Dropdown Option	Pivot reference	left pen, right pen
direction	Dropdown Option	Direction	left forward, left backward, right forward, right backward
radius	Input Value	Circle radius	Real number ≥ 0
unit	Dropdown Option	Distance unit	cm, mm, inch
degree	Input Value	Rotation angle	Real number ≥ 0

JavaScript Code

```
// Left pen, left forward, radius 1cm, 90 degrees | Wait X
__pivot_circle('HamsterS*0', 'left pen', 'left forward',
__getDistance('HamsterS*0', 1, 'cm'), 90, false); // (robot, pivot, direction,
radius, degree, wait_w)

// Left pen, left backward, radius 1cm, 90 degrees | Wait 0
await __pivot_circle('HamsterS*0', 'left pen', 'left backward',
__getDistance('HamsterS*0', 1, 'cm'), 90, true); // (robot, pivot, direction,
radius, degree, wait_w)
```

```

// Left pen, right forward, radius 1cm, 90 degrees | Wait X
__pivot_circle('HamsterS*0', 'left pen', 'right forward',
__getDistance('HamsterS*0', 1, 'cm'), 90, false); // (robot, pivot, direction,
radius, degree, wait_w)

// Left pen, right backward, radius 1cm, 90 degrees | Wait 0
await __pivot_circle('HamsterS*0', 'left pen', 'right backward',
__getDistance('HamsterS*0', 1, 'cm'), 90, true); // (robot, pivot, direction,
radius, degree, wait_w)

// Right pen, left forward, radius 1mm, 90 degrees | Wait X
__pivot_circle('HamsterS*0', 'right pen', 'left forward',
__getDistance('HamsterS*0', 1, 'mm'), 90, false); // (robot, pivot, direction,
radius, degree, wait_w)

// Right pen, left backward, radius 1mm, 90 degrees | Wait 0
await __pivot_circle('HamsterS*0', 'right pen', 'left backward',
__getDistance('HamsterS*0', 1, 'mm'), 90, true); // (robot, pivot, direction,
radius, degree, wait_w)

// Right pen, right forward, radius 1mm, 90 degrees | Wait X
__pivot_circle('HamsterS*0', 'right pen', 'right forward',
__getDistance('HamsterS*0', 1, 'mm'), 90, false); // (robot, pivot, direction,
radius, degree, wait_w)

// Right pen, right backward, radius 1mm, 90 degrees | Wait 0
await __pivot_circle('HamsterS*0', 'right pen', 'right backward',
__getDistance('HamsterS*0', 1, 'mm'), 90, true); // (robot, pivot, direction,
radius, degree, wait_w)

```

Python Code

```
# Left pen, left forward, radius 1cm, 90 degrees | Wait X
__pivot_circle('HamsterS*0', 'left pen', 'left forward',
__getDistance('HamsterS*0', 1, 'cm'), 90, False) # (robot, pivot, direction,
radius, degree, wait_w)

# Left pen, left backward, radius 1cm, 90 degrees | Wait 0
await __pivot_circle('HamsterS*0', 'left pen', 'left backward',
__getDistance('HamsterS*0', 1, 'cm'), 90, True) # (robot, pivot, direction,
radius, degree, wait_w)

# Left pen, right forward, radius 1cm, 90 degrees | Wait X
__pivot_circle('HamsterS*0', 'left pen', 'right forward',
__getDistance('HamsterS*0', 1, 'cm'), 90, False) # (robot, pivot, direction,
radius, degree, wait_w)

# Left pen, right backward, radius 1cm, 90 degrees | Wait 0
await __pivot_circle('HamsterS*0', 'left pen', 'right backward',
__getDistance('HamsterS*0', 1, 'cm'), 90, True) # (robot, pivot, direction,
radius, degree, wait_w)

# Right pen, left forward, radius 1mm, 90 degrees | Wait X
__pivot_circle('HamsterS*0', 'right pen', 'left forward',
__getDistance('HamsterS*0', 1, 'mm'), 90, False) # (robot, pivot, direction,
radius, degree, wait_w)

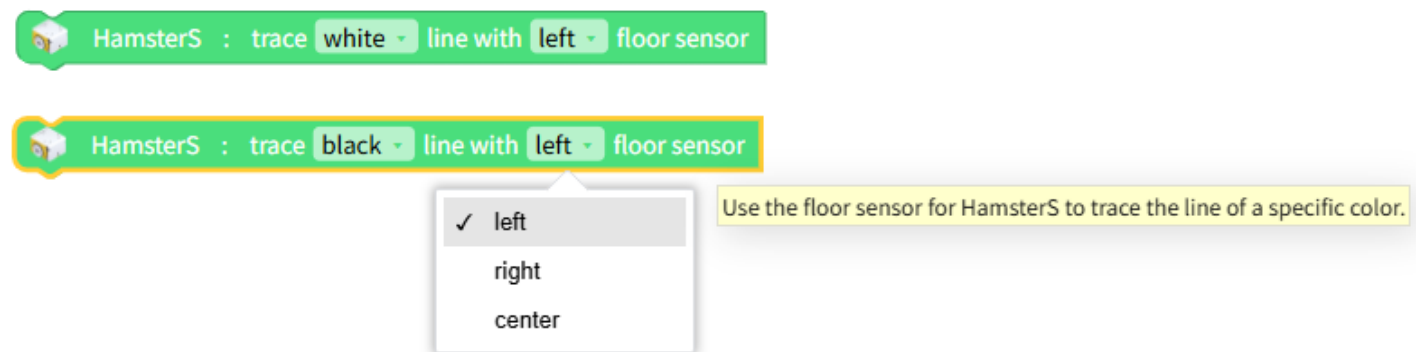
# Right pen, left backward, radius 1mm, 90 degrees | Wait 0
await __pivot_circle('HamsterS*0', 'right pen', 'left backward',
__getDistance('HamsterS*0', 1, 'mm'), 90, True) # (robot, pivot, direction,
radius, degree, wait_w)

# Right pen, right forward, radius 1mm, 90 degrees | Wait X
__pivot_circle('HamsterS*0', 'right pen', 'right forward',
__getDistance('HamsterS*0', 1, 'mm'), 90, False) # (robot, pivot, direction,
radius, degree, wait_w)
```

```
# Right pen, right backward, radius 1mm, 90 degrees | Wait 0
await __pivot_circle('HamsterS*0', 'right pen', 'right backward',
__getDistance('HamsterS*0', 1, 'mm'), 90, True) # (robot, pivot, direction,
radius, degree, wait_w)
```

Follow a Line Using Sensors

Hamster S follows a specific line using the floor sensors.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
direction	Dropdown Option	Floor sensor position	left, right, center
color	Dropdown Option	Line color	black, white

JavaScript Code

```
$('#HamsterS*0:wheel.trace.mode').d = 1; // Follow black line with left floor
sensor
$('#HamsterS*0:wheel.trace.mode').d = 2; // Follow black line with right floor
sensor
$('#HamsterS*0:wheel.trace.mode').d = 3; // Follow black line with center floor
sensor
$('#HamsterS*0:wheel.trace.mode').d = 8; // Follow white line with left floor
sensor
$('#HamsterS*0:wheel.trace.mode').d = 9; // Follow white line with right floor
sensor
```

```
$('HamsterS*0:wheel.trace.mode').d = 10; // Follow white line with center floor sensor
```

Python Code

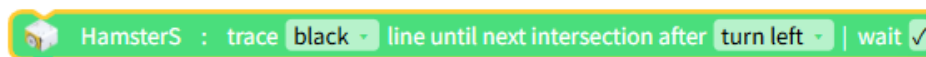
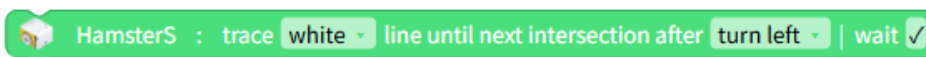
```
__('HamsterS*0:wheel.trace.mode').d = 1 # Follow black line with left floor sensor
__('HamsterS*0:wheel.trace.mode').d = 2 # Follow black line with right floor sensor
__('HamsterS*0:wheel.trace.mode').d = 3 # Follow black line with center floor sensor
__('HamsterS*0:wheel.trace.mode').d = 8 # Follow white line with left floor sensor
__('HamsterS*0:wheel.trace.mode').d = 9 # Follow white line with right floor sensor
__('HamsterS*0:wheel.trace.mode').d = 10 # Follow white line with center floor sensor
```

Turn, Then Follow Line & Stop at the Next Intersection

Hamster S turns in the specified direction, then moves along the line until it reaches the next intersection.

If you check “wait”, it waits until the move is completed.

If you check “wait”, it can only be used inside an async function.



- ✓ turn left
- turn right
- move forward
- uturn

Turn to the assigned direction and move until HamsterS reaches the next intersection. If wait is checked, it will wait until trace completes.

Dropdown Options and Input Values

Name	Type	Description	Range / Type
direction	Dropdown Option	Move direction	turn left, turn right, move forward, uturn
color	Dropdown Option	Line color	black, white

JavaScript Code

```

$('HamsterS*0:wheel.trace.mode').d = 4; // Turn left, follow black line, stop at
next intersection | Wait 0
await $('HamsterS*0:wheel.trace.!mode').w();

$('HamsterS*0:wheel.trace.mode').d = 5; // Turn right, follow black line, stop at
next intersection | Wait X

$('HamsterS*0:wheel.trace.mode').d = 6; // Go forward, follow black line, stop at
next intersection | Wait 0
await $('HamsterS*0:wheel.trace.!mode').w();

$('HamsterS*0:wheel.trace.mode').d = 7; // U-turn, follow black line, stop at
next intersection | Wait X

$('HamsterS*0:wheel.trace.mode').d = 11; // Turn left, follow white line, stop at
next intersection | Wait 0
await $('HamsterS*0:wheel.trace.!mode').w();

$('HamsterS*0:wheel.trace.mode').d = 12; // Turn right, follow white line, stop
at next intersection | Wait X

$('HamsterS*0:wheel.trace.mode').d = 13; // Go forward, follow white line, stop
at next intersection | Wait 0
await $('HamsterS*0:wheel.trace.!mode').w();

$('HamsterS*0:wheel.trace.mode').d = 14; // U-turn, follow white line, stop at
next intersection | Wait X

```

Python Code

```
__('HamsterS*0:wheel.trace.mode').d = 4 # Turn left, follow black line, stop at
next intersection | Wait 0
await __('HamsterS*0:wheel.trace.!mode').w()

__('HamsterS*0:wheel.trace.mode').d = 5 # Turn right, follow black line, stop at
next intersection | Wait X

__('HamsterS*0:wheel.trace.mode').d = 6 # Go forward, follow black line, stop at
next intersection | Wait 0
await __('HamsterS*0:wheel.trace.!mode').w()

__('HamsterS*0:wheel.trace.mode').d = 7 # U-turn, follow black line, stop at next
intersection | Wait X

__('HamsterS*0:wheel.trace.mode').d = 11 # Turn left, follow white line, stop at
next intersection | Wait 0
await __('HamsterS*0:wheel.trace.!mode').w()

__('HamsterS*0:wheel.trace.mode').d = 12 # Turn right, follow white line, stop at
next intersection | Wait X

__('HamsterS*0:wheel.trace.mode').d = 13 # Go forward, follow white line, stop at
next intersection | Wait 0
await __('HamsterS*0:wheel.trace.!mode').w()

__('HamsterS*0:wheel.trace.mode').d = 14 # U-turn, follow white line, stop at
next intersection | Wait X
```

Set Line Tracing Speed

Sets Hamster S line tracing speed.

Speed range is 1 ~ 10.



Set the speed of HamsterS' tracing the line.
The line trace speed ranges from 1 to 10.

Dropdown Options and Input Values

Name	Type	Description	Range / Type
velocity	Input Value	line tracing speed	Integer 1 ~ 10

JavaScript Code

```
$( 'HamsterS*0:wheel.trace.speed' ).d = 1; // Set line tracing speed to 1
```

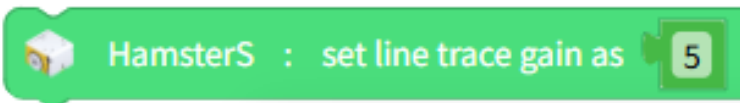
Python Code

```
__( 'HamsterS*0:wheel.trace.speed' ).d = 1 # Set line tracing speed to 1
```

Set Line Tracing Gain

Sets Hamster S line tracing gain.

Gain range is 1 ~ 10.



Set the amount of change in line tracing direction.
The line trace gain ranges from 1 to 10.

Dropdown Options and Input Values

Name	Type	Description	Range / Type
gain	Input Value	line tracing gain	Integer 1 ~ 10

JavaScript Code

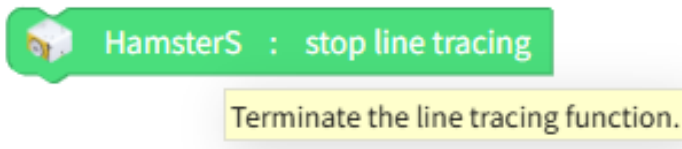
```
$('HamsterS*0:wheel.trace.gain').d = 5; // Set line tracing gain to 5
```

Python Code

```
__('HamsterS*0:wheel.trace.gain').d = 5 # Set line tracing gain to 5
```

Stop Line Tracing

Stops Hamster S line tracing feature.



JavaScript Code

```
$('HamsterS*0:wheel.trace.mode').d = 0; // Stop line tracing
```

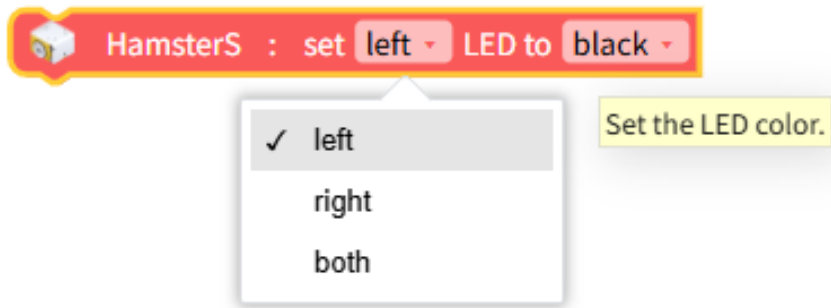
Python Code

```
__('HamsterS*0:wheel.trace.mode').d = 0 # Stop line tracing
```

Set LED Color

Sets Hamster S LED color.

You can set the left, right, or both LEDs.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
direction	Dropdown Option	LED side	left, right, both
color	Dropdown Option	Color	black([0, 0, 0]), red([255, 0, 0]), yellow([255, 255, 0]), green([0, 255, 0]) cyan([0, 255, 255]), blue([0, 0, 255]), magenta([255, 0, 255]), white([255, 255, 255])

JavaScript Code

```
// Set left LED color to red
$('HamsterS*0:led.left').d = [255, 0, 0];

// Set right LED color to green
$('HamsterS*0:led.right').d = [0, 255, 0];

// Set both LED colors to blue
$('HamsterS*0:led.left').d = [0, 0, 255];
$('HamsterS*0:led.right').d = [0, 0, 255];
```

Python Code

```
# Set left LED color to red
__('HamsterS*0:led.left').d = [255, 0, 0]

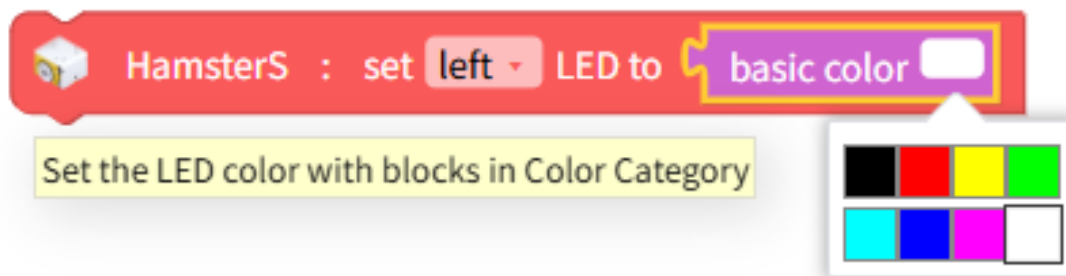
# Set right LED color to green
__('HamsterS*0:led.right').d = [0, 255, 0]

# Set both LED colors to blue
__('HamsterS*0:led.left').d = [0, 0, 255]
__('HamsterS*0:led.right').d = [0, 0, 255]
```

Set LED Color Using Color Category Blocks

Sets Hamster S LED color using blocks from the Color category.

You can set the left, right, or both LEDs.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
direction	Dropdown Option	LED side	left, right, both
color	Input Value	LED color	RGB array

JavaScript Code

```
// Set left LED color to R:255, G:255, B:255
$('HamsterS*0:led.left').d = [255, 255, 255];
```

```
// Set right LED color to R:255, G:255, B:255
$('HamsterS*0:led.right').d = [255, 255, 255];

// Set both LED colors to random colors
$('HamsterS*0:led.left').d = __randomColor();
$('HamsterS*0:led.right').d = __randomColor();
```

Python Code

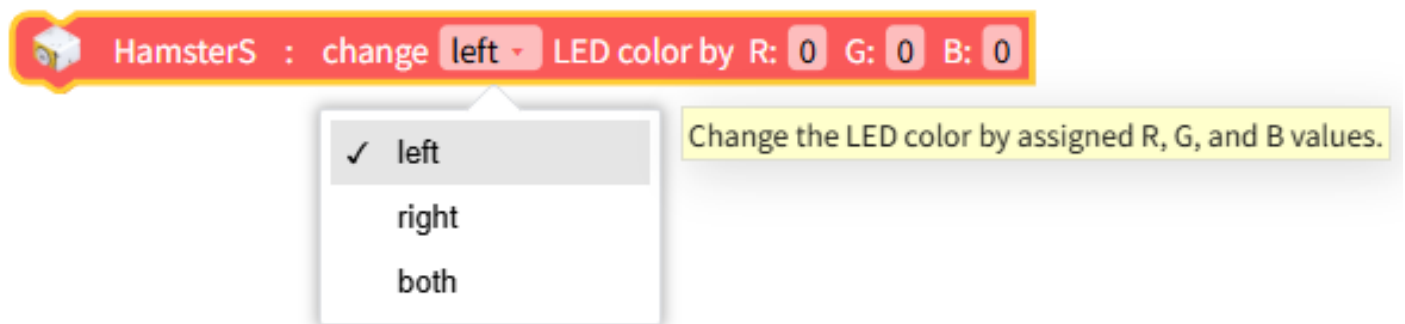
```
# Set left LED color to R:255, G:255, B:255
__('HamsterS*0:led.left').d = [255, 255, 255]

# Set right LED color to R:255, G:255, B:255
__('HamsterS*0:led.right').d = [255, 255, 255]

# Set both LED colors to random colors
__('HamsterS*0:led.left').d = __randomColor()
__('HamsterS*0:led.right').d = __randomColor()
```

Change LED Color by Specified RGB Amount

Changes Hamster S LED color by the specified R, G, B values.
You can change the left, right, or both LEDs.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
direction	Dropdown Option	LED side	left, right, both
color	Input Value	Delta R,G,B values	Integers for each of R, G, B: -255 ~ 255

JavaScript Code

```
// Change left LED color by R:10, G:10, B:10
$('HamsterS*0:led.left').d = [
  $('HamsterS*0:led.left').d[0] + 10,
  $('HamsterS*0:led.left').d[1] + 10,
  $('HamsterS*0:led.left').d[2] + 10];

// Change right LED color by R:100, G:100, B:100
$('HamsterS*0:led.right').d = [
  $('HamsterS*0:led.right').d[0] + 100,
  $('HamsterS*0:led.right').d[1] + 100,
  $('HamsterS*0:led.right').d[2] + 100];

// Change both LED colors by R:-50, G:-50, B:-50
$('HamsterS*0:led.left').d = [
  $('HamsterS*0:led.left').d[0] + -50,
  $('HamsterS*0:led.left').d[1] + -50,
  $('HamsterS*0:led.left').d[2] + -50];
$('HamsterS*0:led.right').d = [
  $('HamsterS*0:led.right').d[0] + -50,
  $('HamsterS*0:led.right').d[1] + -50,
  $('HamsterS*0:led.right').d[2] + -50];
```

Python Code

```
# Change left LED color by R:10, G:10, B:10
__('HamsterS*0:led.left').d = [
  __('HamsterS*0:led.left').d[0] + 10,
  __('HamsterS*0:led.left').d[1] + 10,
  __('HamsterS*0:led.left').d[2] + 10]

# Change right LED color by R:100, G:100, B:100
__('HamsterS*0:led.right').d = [
  __('HamsterS*0:led.right').d[0] + 100,
  __('HamsterS*0:led.right').d[1] + 100,
  __('HamsterS*0:led.right').d[2] + 100]

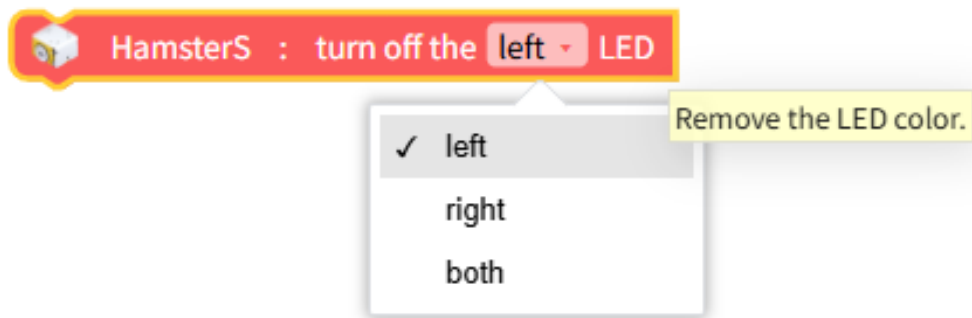
# Change both LED colors by R:-50, G:-50, B:-50
__('HamsterS*0:led.left').d = [
  __('HamsterS*0:led.left').d[0] + -50,
  __('HamsterS*0:led.left').d[1] + -50,
  __('HamsterS*0:led.left').d[2] + -50]
```

```
__('HamsterS*0:led.right').d = [__('HamsterS*0:led.right').d[0] + -50,
__('HamsterS*0:led.right').d[1] + -50, __('HamsterS*0:led.right').d[2] + -50]
```

Turn Off LED

Turns off Hamster S LED color.

You can turn off the left, right, or both LEDs.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
direction	Dropdown Option	LED side	left, right, both

JavaScript Code

```
// Turn off left LED
$('HamsterS*0:led.left').d = [0, 0, 0];

// Turn off right LED
$('HamsterS*0:led.right').d = [0, 0, 0];

// Turn off both LEDs
$('HamsterS*0:led.left').d = [0, 0, 0];
$('HamsterS*0:led.right').d = [0, 0, 0];
```

Python Code

```
# Turn off left LED
__('HamsterS*0:led.left').d = [0, 0, 0]

# Turn off right LED
__('HamsterS*0:led.right').d = [0, 0, 0]

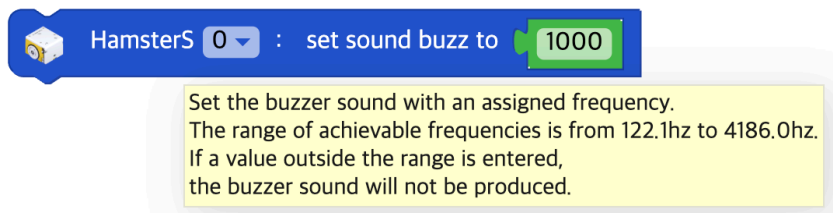
# Turn off both LEDs
__('HamsterS*0:led.left').d = [0, 0, 0]
__('HamsterS*0:led.right').d = [0, 0, 0]
```

Set Buzzer Frequency

Sets Hamster S buzzer sound to the specified frequency.

The range of achievable frequencies is from 122.1hz to 4186.0hz.

If a value outside the range is entered, the buzzer sound will not be produced.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
sound	Input Value	Buzzer frequency	122.1 ~ 4186.0 (hz)

JavaScript Code

```
// Set buzzer frequency to 1000hz
$('HamsterS*0:sound.buzz').d = 1000;
```

Python Code

```
# Set buzzer frequency to 1000hz  
__('HamsterS*0:sound.buzz').d = 1000
```

Play a Note

Hamster S plays the specified musical note.



Play a specific note.

Dropdown Options and Input Values

Name	Type	Description	Range / Type
note	Dropdown Option	Note	Do, Do#, Re, Re#, Mi, Fa, Fa#, So, So#, La, La#, Si
octave	Dropdown Option	Octave	3 ~ 7

JavaScript Code

```
// Play octave 4 Do  
$('HamsterS*0:sound.note').d = 40;  
  
// Play octave 7 Si  
$('HamsterS*0:sound.note').d = 87;
```

Python Code

```
# Play octave 4 Do
__('HamsterS*0:sound.note').d = 40

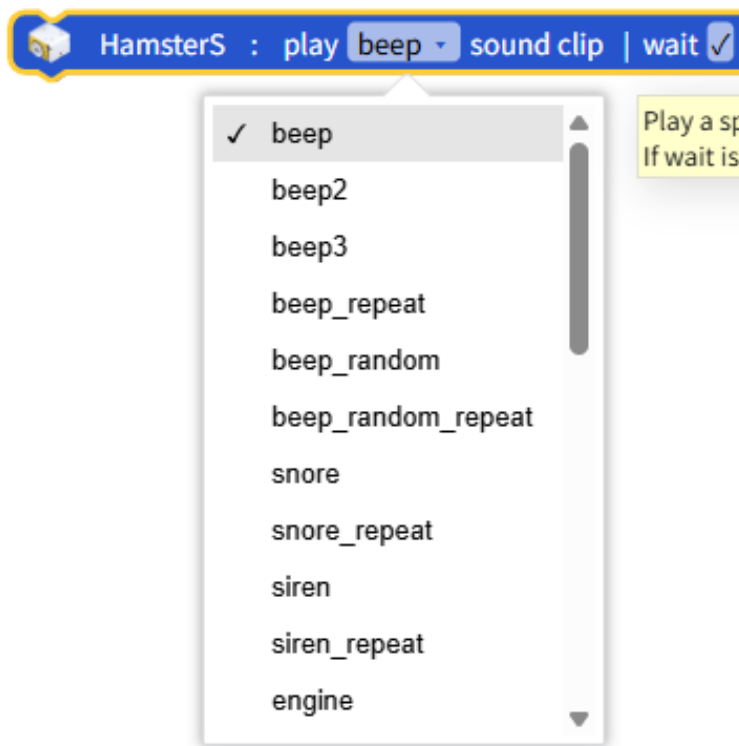
# Play octave 7 Si
__('HamsterS*0:sound.note').d = 87
```

Play Sound Clip

Hamster S plays a specific sound clip.

If you check “wait”, it waits until playback is complete.

If you check “wait”, it can only be used inside an async function.



Play a specific sound clip.
If wait is checked, it will wait until playing sound clip completes.

Dropdown Options and Input Values

Name	Type	Description	Range / Type
sound_clip	Dropdown Option	Sound clip	beep(1), beep2(2), beep3(3), beep_repeat(4), beep_random(5), beep_random_repeat(6), snore(7), snore_repeat(8), siren(9), siren_repeat(10), engine(11), engine_repeat(12), fart_a(13), fart_b(14), noise(15), noise_repeat(16), whistle(17), chop_chop(18), chop_chop_repeat(19), robot(32), dibidibidip(33), melody(34), mission_complete(35), happy(48), angry(49), sad(50), sleep(51), toy_march(52), birthday(53)

JavaScript Code

```
// Play beep | Wait 0
$('HamsterS*0:sound.clip').d = 1;
await $('HamsterS*0:sound.!clip').w();

// Play birthday | Wait X
```

```
$('HamsterS*0:sound.clip').d = 53;
```

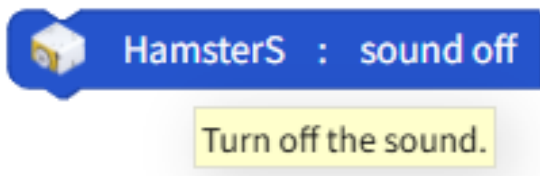
Python Code

```
# Play beep | Wait 0
__('HamsterS*0:sound.clip').d = 1
await __('HamsterS*0:sound.!clip').w()

# Play birthday | Wait X
__('HamsterS*0:sound.clip').d = 53
```

Stop Sound

Stops Hamster S sound.



JavaScript Code

```
// Stop Hamster S sound
__stopSound('HamsterS*0');
```

Python Code

```
# Stop Hamster S sound
__stopSound('HamsterS*0')
```

Is Sound Playing?

Returns whether Hamster S sound is playing as **True(1)** / **False(0)**.

 HamsterS : sound playing?

Returns true if the sound is playing, false if it is not playing.

JavaScript Code

```
// Is Hamster S sound playing? - true if playing, false otherwise
$('HamsterS*0:sound.playing').d;
```

Python Code

```
# Is Hamster S sound playing? - True if playing, False otherwise
__('HamsterS*0:sound.playing').d
```

Wheel Speed Value

Returns the specified Hamster S wheel speed value.

 HamsterS : left wheel speed

✓ left
right

Value of the assigned wheel speed

Dropdown Options and Input Values

Name	Type	Description	Range / Type
direction	Dropdown Option	Direction	left, right

JavaScript Code

```
// Left wheel speed
__getSpeedInput('HamsterS*0', $('HamsterS*0:wheel.speed.left').d);
```

```
// Right wheel speed
__getSpeedInput('HamsterS*0', $('HamsterS*0:wheel.speed.right').d);
```

Python Code

```
# Left wheel speed
__getSpeedInput('HamsterS*0', __('HamsterS*0:wheel.speed.left').d)

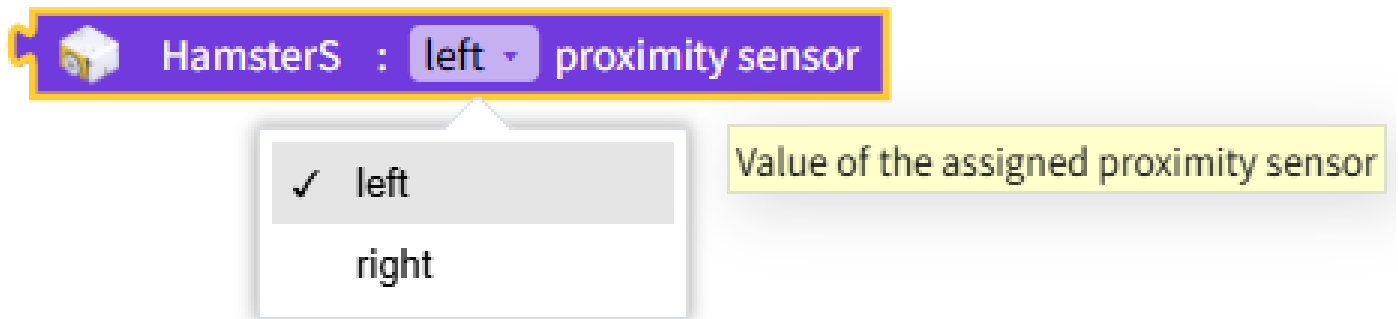
# Right wheel speed
__getSpeedInput('HamsterS*0', __('HamsterS*0:wheel.speed.right').d)
```

Proximity Sensor Value

Returns the specified Hamster S proximity sensor value.

The closer the object is, the larger the value becomes.

The range is 0 to 100.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
direction	Dropdown Option	Direction	left, right

JavaScript Code

```
// Left proximity sensor value
$('HamsterS*0:proximity.left').d;
```

```
// Right proximity sensor value
$('HamsterS*0:proximity.right').d;
```

Python Code

```
# Left proximity sensor value
__('HamsterS*0:proximity.left').d

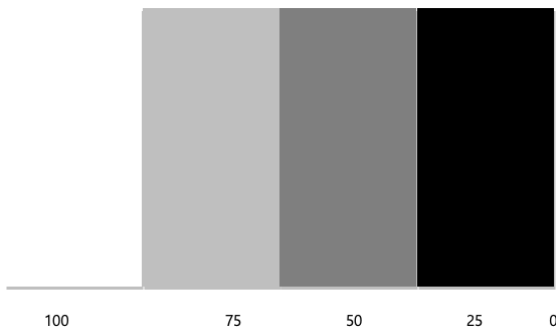
# Right proximity sensor value
__('HamsterS*0:proximity.right').d
```

Floor Sensor Value

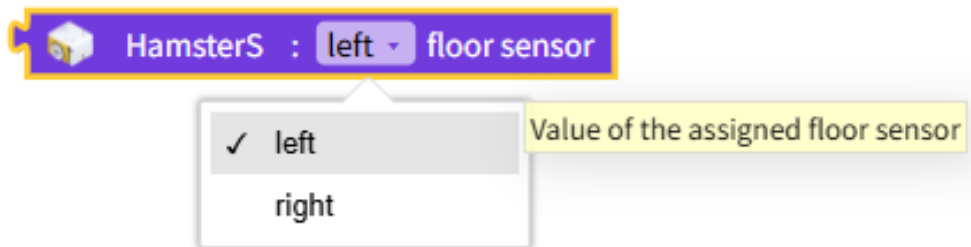
Returns the specified Hamster S floor sensor value.

Black represents the minimum value, and white represents the maximum value.

The range is 0 to 100.



- White is set to 100 based on A4 paper for printers.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
direction	Dropdown Option	Direction	left, right

JavaScript Code

```
// Left floor sensor value
$('HamsterS*0:floor.left').d;

// Right floor sensor value
$('HamsterS*0:floor.right').d;
```

Python Code

```
# Left floor sensor value
__('HamsterS*0:floor.left').d

# Right floor sensor value
__('HamsterS*0:floor.right').d
```

Acceleration Value

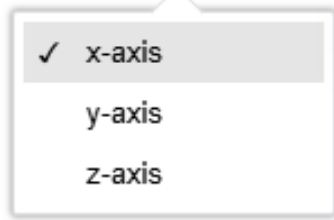
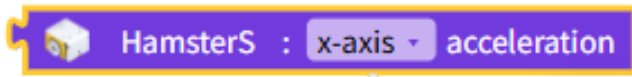
Returns the Hamster S acceleration value for a specific axis.

The accelerometer is a sensor that measures how quickly Hamster S moves or tilts.

The range is -32768 to +32767.

Since Earth's gravity (1g) acts downwards, the Earth's gravity value is Z = -16384.

- * X-axis: Changes when tilting the robot forward or backward.
- * Y-axis: Changes when tilting the robot left or right.
- * Z-axis: Changes when flipping the robot up or down.



Value of the gravitational acceleration sensor on the assigned axis

Dropdown Options and Input Values

Name	Type	Description	Range / Type
axis	Dropdown Option	Axis	x, y, z

JavaScript Code

```
// Acceleration value on x-axis
$('HamsterS*0:acceleration.x').d;

// Acceleration value on y-axis
$('HamsterS*0:acceleration.y').d;

// Acceleration value on z-axis
$('HamsterS*0:acceleration.z').d;
```

Python Code

```
# Acceleration value on x-axis
__('HamsterS*0:acceleration.x').d

# Acceleration value on y-axis
__('HamsterS*0:acceleration.y').d

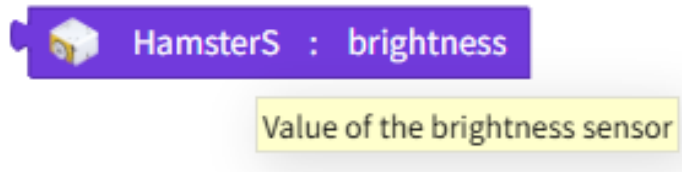
# Acceleration value on z-axis
__('HamsterS*0:acceleration.z').d
```

Light Sensor Value

Returns Hamster S light sensor value.

The value increases when the surroundings are bright, and decreases when it is dark.

The range is 0 to 600.



JavaScript Code

```
// Light sensor value  
$('HamsterS*0:light').d;
```

Python Code

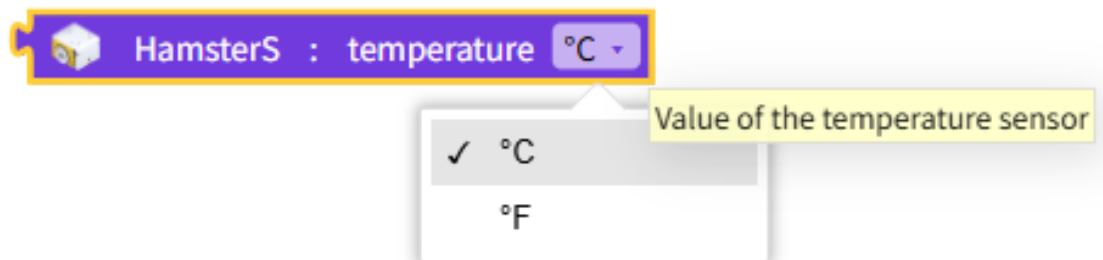
```
# Light sensor value  
__('HamsterS*0:light').d
```

Temperature Sensor Value

Returns Hamster S temperature sensor value.

It represents the internal temperature of the robot, not the room temperature.

The range is -40.0 to +87.5 degrees(°C).



Dropdown Options and Input Values

Name	Type	Description	Range / Type
unit	Dropdown Option	Temperature unit	°C, °F

JavaScript Code

```
// Temperature in Celsius
__getTemperature($('HamsterS*0:temperature').d, '°C');

// Temperature in Fahrenheit
__getTemperature($('HamsterS*0:temperature').d, '°F');
```

Python Code

```
# Temperature in Celsius
__getTemperature(__('HamsterS*0:temperature').d, '°C')

# Temperature in Fahrenheit
__getTemperature(__('HamsterS*0:temperature').d, '°F')
```

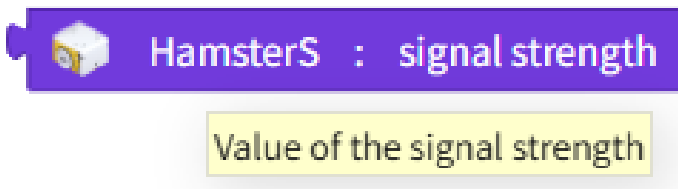
Signal Strength Value

Returns Hamster S signal strength value. It indicates the signal strength between the robot and the dongle.

The value increases as the distance decreases.

For the stable operation, it is recommended to use within the range of -50dBm ~ -80dBm.

If the signal strength value is closer to 0, the signal is stronger. (e.g. -40 > -90)



JavaScript Code

```
// Signal strength value
$('HamsterS*0:signal_strength').d;
```

Python Code

```
# Signal strength value
__('HamsterS*0:signal_strength').d
```

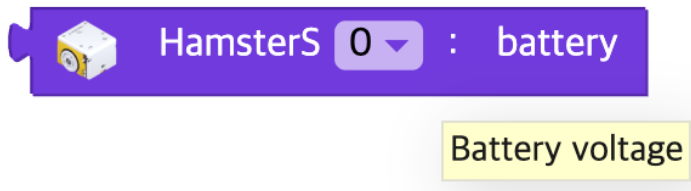
Battery Voltage

Returns Hamster S battery voltage value.

The voltage is approximatedly 4.0V when the battery is fully charged.

It typically operates stably at 3.7V or higher.

Since operating errors may occur if the voltage drops below 3.25V, charging is strongly recommended.



JavaScript Code

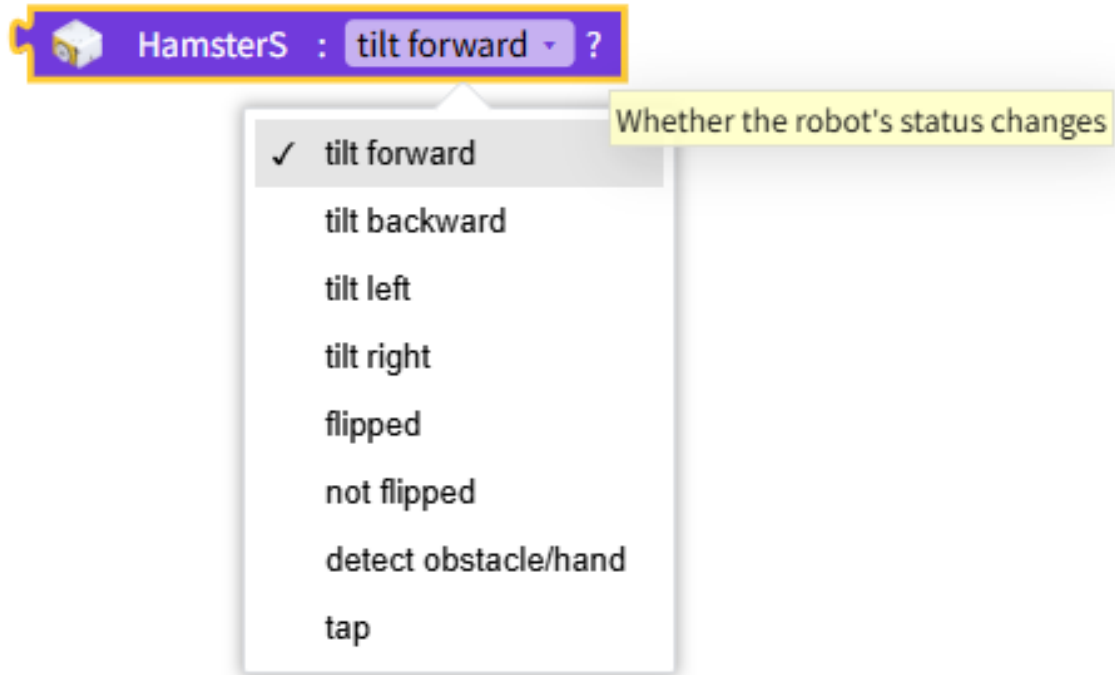
```
// Battery voltage value
$('HamsterS*0:battery.level').d;
```

Python Code

```
# Battery voltage value
__('HamsterS*0:battery.level').d
```

State Change Condition

Depending on the condition, returns whether Hamster S state has changed as **True (1)** / **False (0)**.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
condition	Dropdown Option	Pose / detect condition	tilt forward, tilt backward, tilt left, tilt right, flipped, not flipped, detected obstacle/hand, tap

JavaScript Code

```
// Is Hamster S tilted forward?  
$('HamsterS*0:acceleration.x').d > 5000;  
  
// Is Hamster S tilted backward?  
$('HamsterS*0:acceleration.x').d < -5000;  
  
// Is Hamster S tilted left?
```

```

$('HamsterS*0:acceleration.y').d > 5000;

// Is Hamster S tilted right?
$('HamsterS*0:acceleration.y').d < -5000;

// Is Hamster S flipped over?
$('HamsterS*0:acceleration.z').d > 0;

// Is Hamster S not flipped?
$('HamsterS*0:acceleration.z').d < -3000;

// Did Hamster S detect obstacle/hand?
$('HamsterS*0:proximity.left').d > 50 || $('HamsterS*0:proximity.right').d > 50;

// Was Hamster S tapped?
$('HamsterS*0:acceleration.tap').e;

```

Python Code

```

# Is Hamster S tilted forward?
__('HamsterS*0:acceleration.x').d > 5000

# Is Hamster S tilted backward?
__('HamsterS*0:acceleration.x').d < -5000

# Is Hamster S tilted left?
__('HamsterS*0:acceleration.y').d > 5000

# Is Hamster S tilted right?
__('HamsterS*0:acceleration.y').d < -5000

# Is Hamster S flipped over?
__('HamsterS*0:acceleration.z').d > 0

```

```
# Is Hamster S not flipped?
__('HamsterS*0:acceleration.z').d < -3000

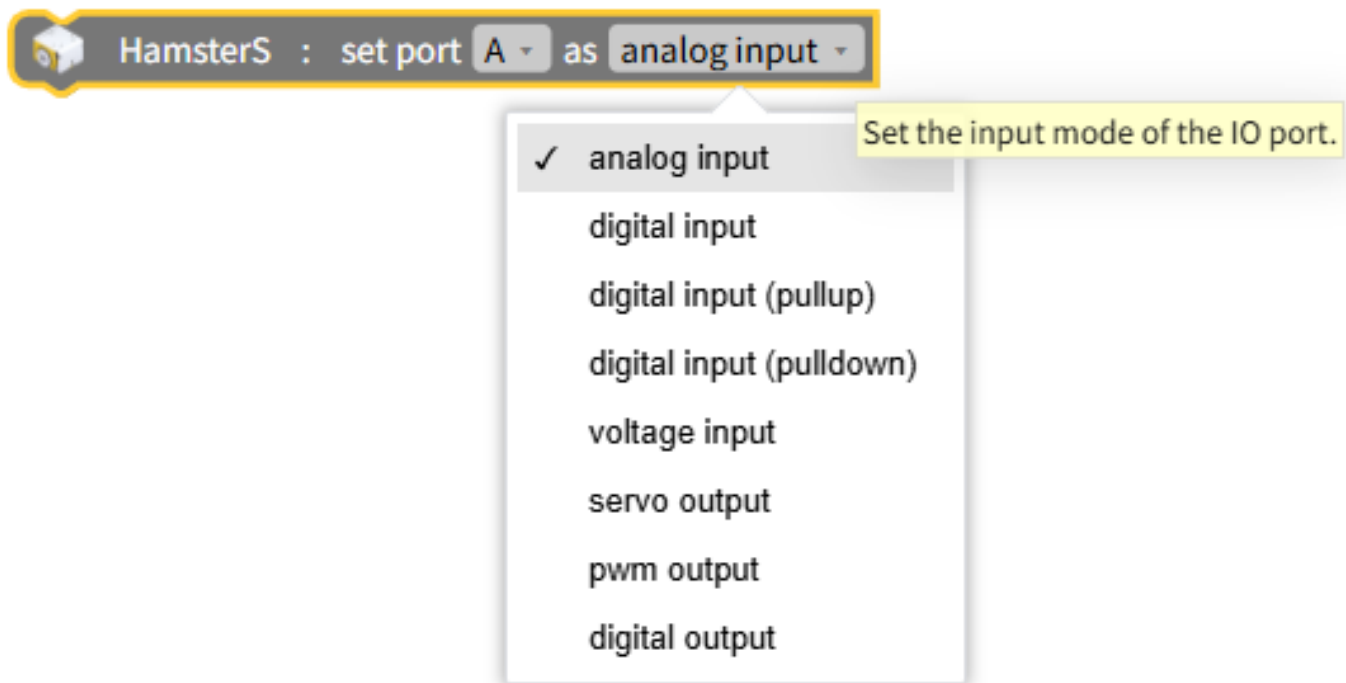
# Did Hamster S detect obstacle/hand?
__('HamsterS*0:proximity.left').d > 50 or __('HamsterS*0:proximity.right').d > 50

# Was Hamster S tapped?
__('HamsterS*0:acceleration.tap').e
```

Set IO Port Input Mode

Sets the input mode of Hamster S I/O ports.

You can set port A, port B, or both ports A and B.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
port	Dropdown Option	I/O port selection	A, B, A and B

Name	Type	Description	Range / Type
mode	Dropdown Option	Port mode	analog input, digital input, digital input(pull-up), digital input(pull-down), voltage input, servo output, pwm output, digital output

JavaScript Code

```
// Set Hamster S port A to analog input
$('HamsterS*0:io.a.mode').d = 0;

// Set Hamster S port A to digital input
$('HamsterS*0:io.a.mode').d = 1;

// Set Hamster S port B to digital input (pull-up)
$('HamsterS*0:io.b.mode').d = 2;

// Set Hamster S port B to digital input (pull-down)
$('HamsterS*0:io.b.mode').d = 3;

// Set Hamster S ports A and B to voltage input
$('HamsterS*0:io.a.mode').d = 4;
$('HamsterS*0:io.b.mode').d = 4;

// Set Hamster S ports A and B to servo output
$('HamsterS*0:io.a.mode').d = 8;
$('HamsterS*0:io.b.mode').d = 8;

// Set Hamster S ports A and B to pwm output
$('HamsterS*0:io.a.mode').d = 9;
$('HamsterS*0:io.b.mode').d = 9;
```

```
// Set Hamster S ports A and B to digital output
$( 'HamsterS*0:io.a.mode' ).d = 10;
$( 'HamsterS*0:io.b.mode' ).d = 10;
```

Python Code

```
# Set Hamster S port A to analog input
__( 'HamsterS*0:io.a.mode' ).d = 0

# Set Hamster S port A to digital input
__( 'HamsterS*0:io.a.mode' ).d = 1

# Set Hamster S port B to digital input (pull-up)
__( 'HamsterS*0:io.b.mode' ).d = 2

# Set Hamster S port B to digital input (pull-down)
__( 'HamsterS*0:io.b.mode' ).d = 3

# Set Hamster S ports A and B to voltage input
__( 'HamsterS*0:io.a.mode' ).d = 4
__( 'HamsterS*0:io.b.mode' ).d = 4

# Set Hamster S ports A and B to servo output
__( 'HamsterS*0:io.a.mode' ).d = 8
__( 'HamsterS*0:io.b.mode' ).d = 8

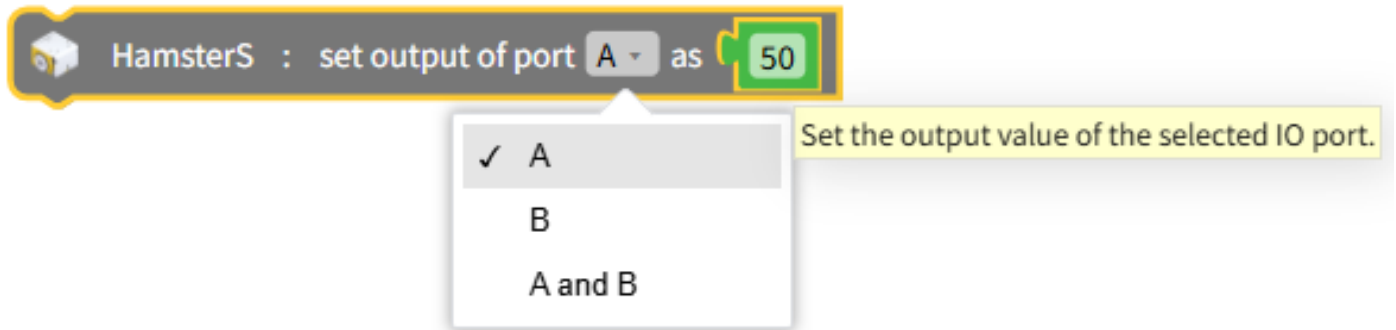
# Set Hamster S ports A and B to pwm output
__( 'HamsterS*0:io.a.mode' ).d = 9
__( 'HamsterS*0:io.b.mode' ).d = 9

# Set Hamster S ports A and B to digital output
__( 'HamsterS*0:io.a.mode' ).d = 10
__( 'HamsterS*0:io.b.mode' ).d = 10
```

Set IO Port Output Value

Sets the output value of Hamster S I/O ports.

You can set port A, port B, or both ports A and B.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
port	Dropdown Option	I/O port selection	A, B, A and B
output	Input Value	Output value	0 ~ 180

JavaScript Code

```
// Set Hamster S port A output to 180
$('HamsterS*0:io.a.out').d = 180;

// Set Hamster S port B output to 0
$('HamsterS*0:io.b.out').d = 0;

// Set Hamster S ports A and B output to 100
$('HamsterS*0:io.a.out').d = 100;
$('HamsterS*0:io.b.out').d = 100;
```

Python Code

```
# Set Hamster S port A output to 180
__('HamsterS*0:io.a.out').d = 180

# Set Hamster S port B output to 0
__('HamsterS*0:io.b.out').d = 0

# Set Hamster S ports A and B output to 100
__('HamsterS*0:io.a.out').d = 100
__('HamsterS*0:io.b.out').d = 100
```

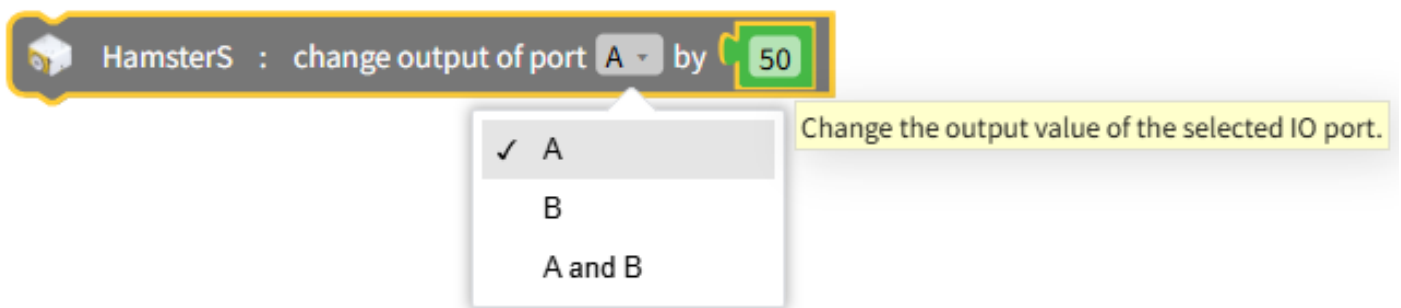
Change IO Port Output Value

Changes the output value of Hamster S I/O ports.

The new output value becomes the current port output value plus the input value.

The new output value is clamped to the range 0 ~ 180.

You can set port A, port B, or both ports A and B.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
port	Dropdown Option	I/O port selection	A, B, A and B
value	Input Value	Delta value	Integer

JavaScript Code

```
// Change Hamster S port A output by +10
$('HamsterS*0:io.a.out').d = $('HamsterS*0:io.a.out').d + 10;
```

```
// Change Hamster S port B output by +20
$('HamsterS*0:io.b.out').d = $('HamsterS*0:io.b.out').d + 20;

// Change Hamster S ports A and B output by +30
$('HamsterS*0:io.a.out').d = $('HamsterS*0:io.a.out').d + 30;
$('HamsterS*0:io.b.out').d = $('HamsterS*0:io.b.out').d + 30;
```

Python Code

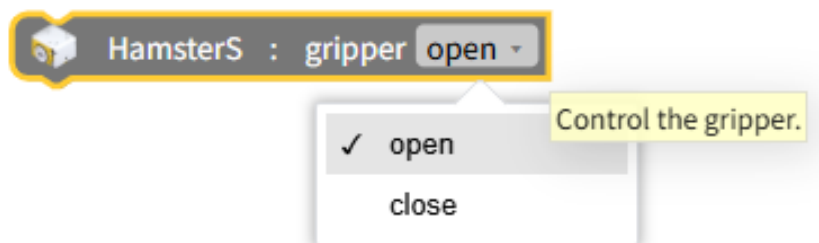
```
# Change Hamster S port A output by +10
__('HamsterS*0:io.a.out').d = __('HamsterS*0:io.a.out').d + 10

# Change Hamster S port B output by +20
__('HamsterS*0:io.b.out').d = __('HamsterS*0:io.b.out').d + 20

# Change Hamster S ports A and B output by +30
__('HamsterS*0:io.a.out').d = __('HamsterS*0:io.a.out').d + 30
__('HamsterS*0:io.b.out').d = __('HamsterS*0:io.b.out').d + 30
```

Open / Close Gripper

Opens or closes Hamster S gripper.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
toggle	Dropdown Option	Gripper toggle	open(1), close(2)

JavaScript Code

```
// Open Hamster S gripper
$('HamsterS*0:io.gripper').d = 1;

// Close Hamster S gripper
$('HamsterS*0:io.gripper').d = 2;
```

Python Code

```
# Open Hamster S gripper
__('HamsterS*0:io.gripper').d = 1

# Close Hamster S gripper
__('HamsterS*0:io.gripper').d = 2
```

Set Shooter Angle

Controls Hamster S by setting the shooter angle.

Angle range is 0 ~ 180.



Set the shooter angle. The angle ranges from 0 to 180.

Dropdown Options and Input Values

Name	Type	Description	Range / Type
angle	Input Value	Shooter angle	Integer 0 ~ 180

JavaScript Code

```
// Set Hamster S shooter angle to 180
$('HamsterS*0:io.shooter').d = 180;
```

```
// Set Hamster S shooter angle to 0
$('HamsterS*0:io.shooter').d = 0;
```

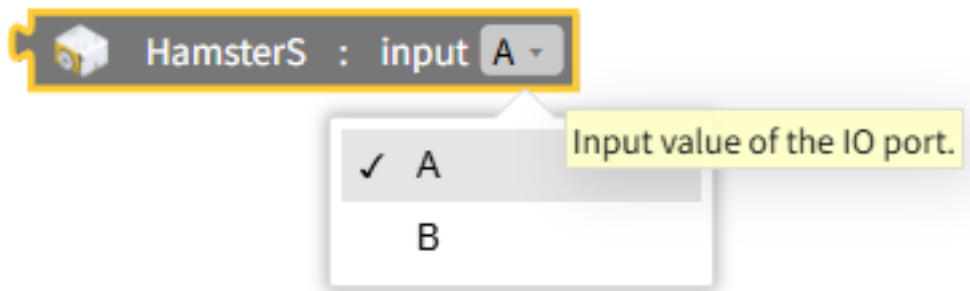
Python Code

```
# Set Hamster S shooter angle to 180
__('HamsterS*0:io.shooter').d = 180

# Set Hamster S shooter angle to 0
__('HamsterS*0:io.shooter').d = 0
```

IO Port Input Value

Returns Hamster S I/O port input value.



Dropdown Options and Input Values

Name	Type	Description	Range / Type
port	Dropdown Option	I/O port	A, B

JavaScript Code

```
// Hamster S port A input value
$('HamsterS*0:io.a.in').d;
```

```
// Hamster S port B input value
$('HamsterS*0:io.b.in').d;
```

Python Code

```
# Hamster S port A input value
__('HamsterS*0:io.a.in').d

# Hamster S port B input value
__('HamsterS*0:io.b.in').d
```